

```

class Regex5
{
    public static void main(String args[])
    {
        Pattern pat = Pattern.compile("S.*r");
        String str = "Sachin Tendulkar Hits a Sixer";
        Matcher mat = pat.matcher(str);
        int i=1;
        while(mat.find() )
        {
            System.out.println("Matched String " + i +
" : " + mat.group( ));
            i++;
        }
    }
}

```

On execution, the program *regex5.java* displays the message 'Matched String 1 : Sachin Tendulkar Hits a Sixer' on the terminal. What is the reason for matching the whole string? Because the pattern 'S.*r' searches for a string starting with S, followed by zero or more occurrences of any character, and finally ending with an r. Since the pattern '.*' results in a greedy match, the whole string is matched.

Now replace the line of code:

```
'Pattern pat = Pattern.compile("S.*r");'
```

...in *Regex5.java* with the line:

```
'Pattern pat = Pattern.compile("S.*?r");'
```

...to get *Regex6.java*. What will be the output of *Regex6.java*? Since this is the last article of this series on regular expressions, I request you to try your best to find the answer before proceeding any further. Figure 5 shows the output of *Regex5.java* and *Regex6.java*. But what is the reason for the output shown by *Regex6.java*? Again, I request you to ponder over the problem for some time and find out the answer. If you don't get the answer, download the file *Regex6.java* from the link shown earlier, and in that file I have given the explanation as a comment.

So, with that example, let us wind up our discussion about regular expressions in Java. Java is a very powerful programming language and the effective use of regular expressions will make it even more powerful. The basic stuff discussed here will definitely kick-start your journey towards the efficient use of regular expressions in Java. And now it is time to say farewell.

In this series we have discussed regular expression processing in six different programming languages. Four of these—Python, Perl, PHP and Java—use a regular expression style called PCRE (Perl Compatible Regular Expressions). The other two programming languages we discussed in

```

[root@deepu CODE]# javac Regex5.java
[root@deepu CODE]# java Regex5
Matched String 1 : Sachin Tendulkar Hits a Sixer
[root@deepu CODE]# javac Regex6.java
[root@deepu CODE]# java Regex6
Matched String 1 : Sachin Tendulkar
Matched String 2 : Sixer

```

Figure 5: Output of *Regex5.java* and *Regex6.java*

this series, C++ and JavaScript, use a style known as the ECMAScript regular expression style. The articles in this series were never intended to describe the complexities of intricate regular expressions in detail. Instead, I tried to focus on the different flavours of regular expressions and how they can be used in various programming languages. Any decent textbook on regular expressions will give a language-agnostic discussion of regular expressions but we were more worried about the actual execution of regular expressions in programming languages.

Before concluding this series, I would like to go over the important takeaways. First, always remember the fact that there are many different regular expression flavours. The differences between many of them are subtle, yet they can cause havoc if used indiscreetly. Second, the style of regular expression used in a programming language depends on the flavour of the regular expression implemented by the language's regular expression engine. Due to this reason, a single programming language may support multiple regular expression styles with the help of different regular expression engines and library functions. Third, the way different languages support regular expressions is different. In some languages the support for regular expressions is part of the language core. An example for such a language is Perl. In some other languages the regular expressions are supported with the help of library functions. C++ is a programming language in which regular expressions are implemented using library functions. Due to this, all the versions and standards of some programming languages may not support the use of regular expressions. For example, in C++, the support for regular expressions starts with the C++11 standard. For the same reason, the different versions of a particular programming language itself might support different regular expression styles. You must be very careful about these important points while developing programs using regular expressions to avoid dangerous pitfalls.

So, finally, we are at the end of a long journey of learning regular expressions. But an even longer and far more exciting journey of practising and developing regular expressions lies ahead. Good luck! 

By: Deepu Benson

The author is a free software enthusiast whose area of interest is theoretical computer science. He maintains a technical blog at www.computingforbeginners.blogspot.in and can be reached at deepumb@hotmail.com.